Guia de uso do Docker

Como subir o banco de dados do Unishop na sua máquina

Projeto Unishop · Desenvolvimento de Plataformas Web (Unifor, T242-12)

Guia para os membros da equipe: Pedro, Felipe, Ana Clara e Francisco.

Em uma frase: o Docker roda o PostgreSQL (nosso banco) dentro de uma "caixinha" isolada. Você não instala o PostgreSQL no Windows; só liga e desliga a caixinha. A API e o frontend continuam rodando normalmente com `npm run dev`.

Índice

- 1. O que é o Docker e por que usamos
- 2. Instalação
- 3. Conceitos básicos
- 4. Passo a passo: primeira vez no projeto
- 5. Rotina do dia a dia
- 6. Comandos úteis (cola)
- 7. Problemas comuns e como resolver
- 8. Perguntas frequentes

1. O que é o Docker e por que usamos

Sem o Docker, cada pessoa teria de instalar o PostgreSQL na própria máquina, na mesma versão, com o mesmo usuário e a mesma senha. Isso dá erro em uma máquina e funciona em outra. Com o Docker, o projeto traz um arquivo (`docker-compose.yml`) que descreve o banco, e todo mundo sobe **exatamente o mesmo**.

No Unishop, o Docker cuida **só do banco**. A API (Express) e o frontend (Vite) rodam direto na sua máquina.

Parte	Onde roda	Como liga
Banco PostgreSQL 18	Docker (container db)	docker compose up -d
API (backend)	Sua máquina, porta 3000	npm run dev (pasta backend)
Frontend	Sua máquina, porta 5173	npm run dev (pasta frontend)

2. Instalação

Instale o **Docker Desktop** (gratuito para estudantes) em docker.com. No Windows ele usa o WSL 2; o instalador oferece ativar. Depois de instalar:

- Reinicie o computador se o instalador pedir.
- Abra o **Docker Desktop** e espere o ícone da baleia ficar fixo (sem animação). Isso significa que o Docker está ligado.
- Confirme no terminal:

```
docker --version
docker compose version
```

Importante: o Docker Desktop precisa estar **aberto** sempre que você for usar o banco. Se o terminal disser que não consegue conectar ao Docker, quase sempre é porque ele está fechado.

3. Conceitos básicos

Termo	O que significa	No Unishop
Imagem	Receita pronta de um programa (o "molde").	postgres:18-alpine
Container	A imagem em execução (a "caixinha" ligada).	O serviço db
Volume	Pasta onde o container guarda dados para não perdê-los ao desligar.	unishop_dev_data (os dados do banco)
Porta	Número pelo qual sua máquina fala com o container.	5432 (somente 127.0.0.1)
Compose	Arquivo que descreve os containers do projeto.	docker-compose.yml na raiz

Dados de conexão do banco de desenvolvimento (definidos no `docker-compose.yml`):

Item	Valor
Usuário	dev
Senha	dev
Banco	unishop_dev
Endereço	localhost:5432
DATABASE_URL	postgresql://dev:dev@localhost:5432/unishop_dev

Essa senha "dev" é só para desenvolvimento local. O banco só aceita conexões da sua própria máquina (127.0.0.1) e nunca é usado em produção (lá é o Supabase).

4. Passo a passo: primeira vez no projeto

Faça uma vez, na ordem. Os comandos de Docker rodam na **raiz** do projeto (onde está o docker-compose.yml); os de npm, dentro da pasta **backend**.

Passo 1 — Atualize o código

```
git pull
```

Passo 2 — Suba o banco

```
docker compose up -d
```

A primeira vez baixa a imagem do PostgreSQL (pode demorar alguns minutos). O `-d` deixa o banco rodando em segundo plano. Confira:

```
docker compose ps
```

A linha do serviço `db` deve mostrar o estado **running** (ou "Up").

Como o container é criado e iniciado

Você **não cria o container à mão**: não existe `docker run` para escrever. O arquivo `docker-compose.yml`, na raiz do projeto, já descreve tudo (imagem, usuário, senha, porta e volume). O `docker compose up -d` lê esse arquivo e faz, nesta ordem:

- baixa a imagem `postgres:18-alpine` (só na primeira vez);
- cria o volume `unishop_dev_data`, onde os dados do banco ficam guardados;
- cria o container `db` a partir da imagem;
- liga o container, e o PostgreSQL inicia dentro dele (na primeira vez, cria o banco `unishop_dev`).

Para ter certeza de que subiu sem erro, veja as mensagens do banco:

```
docker compose logs db
```

Procure a frase **"database system is ready to accept connections"**. Se ela aparecer, o banco está pronto. Se aparecer "error", copie a mensagem e consulte a seção 7.

Comando	O que acontece	Quando usar
docker compose up -d	Cria o container (se não existir) e liga.	Primeira vez e todo começo de dia.
docker compose start	Só religa um container que já existe e foi parado.	Depois de um stop.
docker compose stop	Desliga o container; ele continua existindo e os dados ficam salvos.	Fim do dia.
docker compose down	Desliga e **remove** o container; o volume (dados) é mantido.	Quando quiser recriar o container.

Onde rodar: os comandos `docker compose ...` só funcionam na pasta que contém o `docker-compose.yml` (a **raiz** do Unishop, não a pasta backend). Se aparecer "no configuration file provided", você está na pasta errada.

Passo 3 — Crie o arquivo .env do backend

```
cd backend
copy .env.example .env
```

Armadilha conhecida: no `.env` deixe `DATABASE_URL_UNPOOLED` **igual ao DATABASE_URL** (ou apague a linha). Se ela ficar vazia, o Prisma acusa "Connection url is empty". Preencha também o `JWT_SECRET` com um texto longo e o `GOOGLE_CLIENT_ID` se for testar o login com Google. O .env nunca vai para o Git.

Passo 4 — Instale as dependências e gere o Prisma Client

```
npm install
npx prisma generate
```

O `prisma generate` cria a pasta `src/generated` (código gerado, ignorado pelo Git). Cada pessoa gera a sua; é normal ela não vir no repositório.

Passo 5 — Crie as tabelas

```
npx prisma migrate deploy
```

Aplica as migrações `0_init` e `1_regras_manuais` (as 8 tabelas, restrições, índices e a view). Use `migrate deploy` para só aplicar o que já existe. O `npm run db:migrate` serve para criar migração nova; evite no dia a dia.

Passo 6 — Popule o básico (seed)

```
npm run db:seed
```

Cria as 6 categorias e a conta de administrador (usa `ADMIN_EMAIL` e `ADMIN_PASSWORD` do .env).

Passo 7 — Rode o projeto

```
# terminal 1, pasta backend
npm run dev

# terminal 2, pasta frontend
npm run dev
```

Abra `http://localhost:5173`. Para testar o backend, rode `npm test` na pasta backend (os testes usam repositórios em memória e não tocam no seu banco).

5. Rotina do dia a dia

Quando	O que fazer
Começar a trabalhar	1) Abrir o Docker Desktop. 2) docker compose up -d. 3) npm run dev (backend e frontend).
Terminar o dia	Parar os npm run dev com Ctrl+C. O banco pode ficar ligado ou: docker compose stop.
Alguém criou migração nova	git pull, depois npx prisma migrate deploy (na pasta backend).
Mudou o schema.prisma	npx prisma generate (e migrate, se houver migração nova).
Quer o banco do zero	npm run db:reset (apaga TODOS os dados locais e refaz migrações e seed).

6. Comandos úteis (cola)

Comando	Para que serve
docker compose up -d	Liga o banco em segundo plano.
docker compose ps	Mostra se o banco está rodando.
docker compose logs db	Mostra as mensagens do banco (útil para erros).
docker compose stop	Desliga o banco; os dados continuam salvos.
docker compose start	Religa um banco que foi parado.
docker compose down	Remove o container; os dados continuam no volume.
docker compose down -v	**CUIDADO:** remove também o volume, ou seja, apaga todos os dados do banco local.
docker compose exec db psql -U dev -d unishop_dev	Abre o terminal SQL dentro do banco (sair: \q).
npx prisma studio	Abre uma tela no navegador para ver e editar as tabelas.

Regra de ouro: `stop` e `down` são seguros. Só o `down -v` e o `db:reset` apagam dados. No banco local não há problema, mas só faça se for de propósito.

7. Problemas comuns e como resolver

Sintoma	Causa provável	Solução
"Cannot connect to the Docker daemon" ou erro de pipe	Docker Desktop fechado.	Abra o Docker Desktop e espere a baleia ficar parada.
"port is already allocated" (5432)	Outro PostgreSQL (instalado no Windows) usa a porta.	Pare o outro serviço PostgreSQL ou desinstale; depois docker compose up -d.
P1001: Can't reach database server	Banco desligado ou porta errada.	docker compose ps; se parado, docker compose up -d. Confira o DATABASE_URL.
"Connection url is empty"	DATABASE_URL_UNPOOLED vazio no .env.	Deixe igual ao DATABASE_URL ou apague a linha.
Erro de autenticação (password)	Volume antigo criado com outra senha.	Se não houver dados importantes: docker compose down -v e subir de novo.
"Cannot find module .../generated/prisma"	Esqueceu o prisma generate.	npx prisma generate na pasta backend.
Tabelas não existem	Migrações não aplicadas.	npx prisma migrate deploy e depois npm run db:seed.
Falha de CORS no navegador	FRONTEND_URL não inclui a porta do front.	No .env, coloque as origens separadas por vírgula (ex.: 5173 e 5174) e reinicie a API.
Docker lento ou "WSL" com erro	WSL 2 desatualizado.	No PowerShell (admin): wsl --update e reinicie.

8. Perguntas frequentes

Preciso mexer em algo do Docker para programar? Não. Depois do passo a passo, é só ligar o banco e usar o npm normalmente.

Meus dados ficam salvos se eu desligar o computador? Sim. Ficam no volume `unishop_dev_data`.

Cada pessoa tem o seu banco? Sim. É local; o que um cadastra não aparece para os outros. O que é compartilhado é o schema (via migrações no Git) e o seed.

E em produção? Não usa esse Docker: o banco é o PostgreSQL do Supabase, e a API roda na Vercel (ver docs/deploy/deploy.md).

O que é o docker-compose.test.yml? Um segundo banco, só para testes, na porta 5433 e que perde os dados ao desligar. Hoje o `npm test` não depende dele.

Documentos relacionados: docs/BancoDeDados/bd.md (banco, migrações, papéis) e docs/deploy/deploy.md.